Relatório de Auditoria de Segurança

Botezini ERP

28 de agosto de 2026

1	2	2	1
CRÍTICA	ALTA	MÉDIA	BAIXA

Escopo auditado

Todo o código-fonte da aplicação (**src/**, 268 arquivos TypeScript/TSX), todas as 46 migrações SQL em **supabase/**, o dump do schema de produção versionado no repositório, os scripts em **scripts/**, os arquivos de ambiente e o histórico completo do Git.

Auditoria estática, feita sobre o código e sobre o dump do schema de produção versionado no repositório. Nenhum teste foi executado contra o banco em produção — os achados que dependem do estado real do banco estão marcados com essa condição.

Nota metodológica

Cada uma das cinco categorias pedidas foi traduzida para o equivalente desta stack antes de procurar qualquer coisa. Categoria que não se aplica está dita como tal, em vez de preenchida à força.

Stack detectada

SPA React 18 + TypeScript + Vite + Tailwind v4. **Não existe backend próprio**: o navegador fala direto com o PostgREST do Supabase usando a chave anônima. Autenticação: Supabase Auth. Sem Docker, sem CI, sem Helm ou Terraform. Deploy pela Vercel.

Mecanismo de isolamento

RLS do PostgreSQL, em modelo single-tenant. Não há filtro por organização: é uma empresa só, e a separação acontece por PAPEL — `admin`, `manager` e `factory` — através das funções `is_admin_or_manager()` e `is_factory()`. A auditoria mediu se essa tranca existe e se fecha.

C1 — Banco sem tranca

Mapeado para: tabelas sem RLS habilitada, políticas permissivas demais, e políticas antigas que sobrevivem a uma migração de endurecimento. **Políticas de RLS são combinadas por OU** — uma política frouxa esquecida anula todas as rigorosas.

C2 — Permissão no navegador

Cada porta que o frontend fecha por papel foi cruzada com o equivalente no servidor: política de RLS ou verificação dentro da função RPC.

C3 — IDOR

Sem rotas de backend, o equivalente são as **55 funções SECURITY DEFINER** — que ignoram RLS por definição — e o acesso ao Storage por caminho. Todas foram inspecionadas, uma a uma, não por amostragem.

C4 — Chaves expostas

Código-fonte, SQL, scripts, arquivos de ambiente, o bundle compilado e o histórico completo do Git.

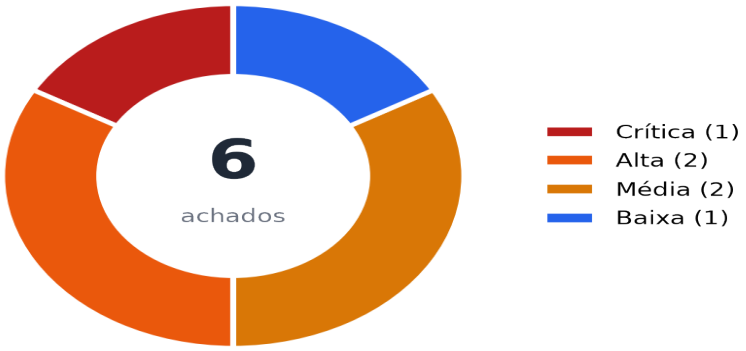
C5 — XSS

Busca por todos os sinks do React e do DOM, por URLs controladas pelo usuário em `href`/`src`, e pelo caminho de upload de arquivos.

Resumo executivo

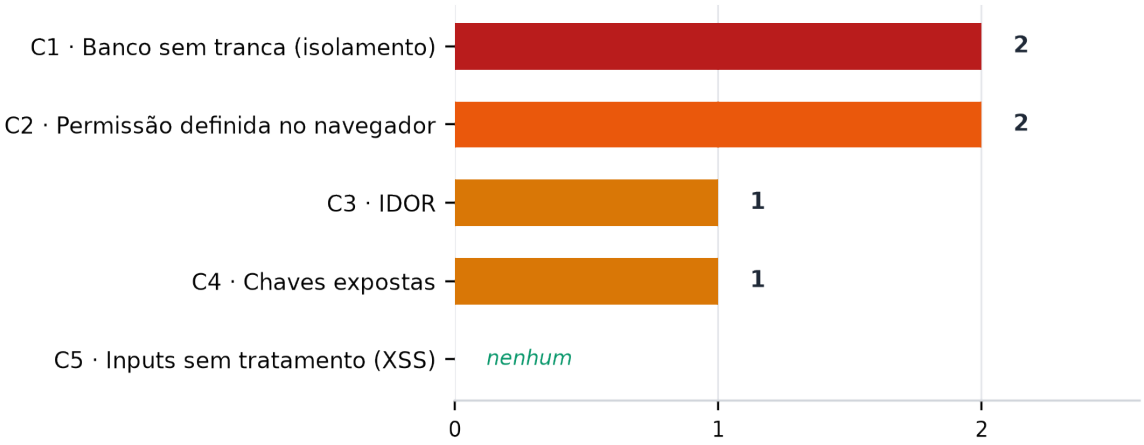
Foram encontrados **6 achados** e registrados **9 pontos fortes** verificados. O sistema tem uma base de segurança sólida — RLS em todas as tabelas, papel anônimo revogado, nenhum sink de XSS, e verificação de autorização em 50 das 55 funções privilegiadas. O risco central não é ausência de proteção: é **uma política antiga que sobreviveu a uma migração de endurecimento** e, por ser combinada por OU, anula a proteção nova.

Achados por severidade



Achados por categoria

A barra usa a cor da *pior* severidade da categoria.



Pontos fortes verificados

O que foi conferido e está correto. Serve como prova de cobertura da auditoria: estes pontos foram procurados e encontrados em ordem.

O que está protegido	Evidência
Nenhum sink de XSS no frontend inteiro	Busca em 268 arquivos: zero ocorrências de `dangerouslySetInnerHTML`, `.innerHTML`, `eval()` ou `new Function()`. Todo texto vindo do usuário passa pelo escape automático do JSX. Não há renderização de Markdown nem de HTML.
URL de WhatsApp construída à prova de esquema perigoso	`src/utils/mensagensCliente.ts:457-462` — o telefone é reduzido a dígitos (`replace(/\D/g,'')`), o texto passa por `encodeURIComponent` e o endereço é fixo em `https://wa.me/`. Um `javascript:` não tem por onde entrar.
Cadastro de usuário não aceita papel vindo do cliente	`security_hardening_single_tenant.sql:66-80` — `handle_new_user()` grava `'factory'` literal, ignorando `raw_user_meta_data`. E a coluna tem `DEFAULT 'factory'`. Fecha a escalada mais comum em Supabase: pedir `role: admin` no cadastro.
A política de edição de perfil foi escrita corretamente	`perfil_nome_e_foto.sql:39-47` prende o cargo no `WITH CHECK`, comparando com o valor atual. O raciocínio está certo e documentado no próprio arquivo — ela só não tem efeito porque a política antiga sobreviveu (achado nº 1).
RLS habilitada em 30 de 30 tabelas de produção	Conferido no dump versionado: toda tabela do schema `public` aparece com `ENABLE ROW LEVEL SECURITY`. As tabelas criadas depois (`leads`, `client_contacts`, `consumo_nao_apurado`, `rendimento_do_corte`, `producao_baixas`) trazem RLS e política nas próprias migrações.
50 de 55 funções SECURITY DEFINER verificam autorização	Todas as funções financeiras que escrevem — `pay_obligation`, `reverse_payment`, `cancel_obligation`, `set_bank_balance`, `recalculate_all_bank_balances`, `move_envelope`, `transfer_envelope` — abrem com `assert_finance_access()`. As de produção verificam `is_admin_or_manager() OR is_factory()`. As cinco sem checagem são gatilhos internos ou o auxiliar `current_user_role()`, exceto o achado nº 2.
Papel anônimo revogado de tudo	`security_hardening_single_tenant.sql:83-86` — `REVOKE ALL` em tabelas, sequências e funções para `anon`. Sem sessão, o banco não responde nada.
A chave no bundle é a anônima, como deve ser	`src/config/env.ts:21-22` lê apenas `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY`. Nenhuma `service_role` no código, no bundle ou no histórico. Sem valores embutidos: ausência de chave apenas liga o Modo Restrito, não cai num padrão inseguro.
Nenhum IDOR nas funções RPC	As 55 funções SECURITY DEFINER foram lidas uma a uma. As que recebem id — `informar_gasto_do_corte`, `resolver_consumo_nao_apurado`, `reverse_payment`, `cancel_obligation` — verificam papel antes de agir. No modelo single-tenant, papel é o critério correto: não existe dono por linha a conferir.

Pontos fracos — os riscos centrais

Uma política esquecida anula todo o endurecimento

O risco central. O sistema foi migrado de um modelo por dono (`auth.uid() = user_id`) para um modelo por papel, mas a migração de endurecimento só ACRESCENTA políticas — nunca derruba as antigas. Como políticas de RLS são combinadas por OU, as antigas continuam valendo em paralelo. Em `profiles` isso é escalada de privilégio; nas demais tabelas é permissividade extra que ninguém revisou.

A separação de papéis existe na tela, não sempre no banco

O frontend fecha o Financeiro para o papel `factory` em três pontos de `App.tsx`. No servidor, quase tudo tem o equivalente — mas as duas exceções encontradas (achados 2 e 3) são justamente as que dão acesso a dinheiro.

A superfície de segurança do Storage não está no repositório

Nenhuma das 46 migrações versiona política de bucket. Quem revisar o código não tem como saber se os anexos financeiros estão protegidos, e um ajuste feito no painel não deixa rastro em revisão nem em histórico.

Achados detalhados

C1 — Banco sem tranca (isolamento)

CRÍTICA **#1 — Qualquer usuário pode se promover a administrador**

`supabase/schema_live.sql` `npx:2103 (dump de produção)`

```
CREATE POLICY "owner" ON "public"."profiles"
  USING (("auth"."uid"() = "id"))
  WITH CHECK (("auth"."uid"() = "id"))
```

POR QUE É EXPLORÁVEL

A política não tem cláusula `FOR`, então vale para SELECT, INSERT, UPDATE e DELETE. A única condição de escrita é que a linha seja a sua — a coluna `role` fica sem restrição nenhuma.

A migração `perfil_nome_e_foto.sql:39-47` cria uma política correta, que prende o cargo no `WITH CHECK`. Mas **políticas de RLS são combinadas por OU**: basta uma permitir. A política `owner` nunca é removida — nenhuma das 46 migrações a derruba, e `security_hardening_single_tenant.sql` só ACRESCENTA políticas.

Exploração, do console do navegador, com uma conta `factory`:
`await supabase.from('profiles').update({role:'admin'}).eq('id', meuId)`

A partir daí `is_admin_or_manager()` passa a devolver verdadeiro, e todo o Financeiro, todos os dados e todas as RPCs se abrem.

IMPACTO

Escalada completa de privilégio, do papel mais baixo ao de administrador. Dá acesso a saldos bancários, contas a pagar e receber, cadastro de clientes e à capacidade de apagar dados.

CONDIÇÃO DE EXPLORABILIDADE

Requer uma conta autenticada de qualquer papel. Nenhuma configuração especial. A chave anônima necessária está publicada no bundle, por desenho.

CORREÇÃO

Derrubar a política antiga e verificar que nenhuma outra permissiva sobrou:

```
DROP POLICY IF EXISTS "owner" ON public.profiles;
```

Depois conferir com `SELECT policyname, cmd, qual, with_check FROM pg_policies WHERE tablename='profiles';` — devem sobrar apenas as duas de `perfil_nome_e_foto.sql` e a de admin/manager.

Como defesa em profundidade, um gatilho `BEFORE UPDATE` em `profiles` que rejeite mudança de `role` quando `auth.uid() = NEW.id`: a promoção passa a exigir outra pessoa, independentemente de qual política venha a existir no futuro.

BAIXA **#6 — Tabelas de almoxarifado abertas a qualquer usuário autenticado**

`supabase/schema_live.sqlnpx (dump de produção) — 5 políticas`

```
CREATE POLICY "authenticated" ON "public"."wms_items"          USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_categories"     USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_item_variations" USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_stock_items"    USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_stock_movements" USING ((auth.uid() IS NOT NULL))
```

POR QUE É EXPLORÁVEL

`auth.uid() IS NOT NULL` é verdadeiro para **todo** usuário logado. A RLS está habilitada — o que faz a tabela parecer protegida numa auditoria superficial — mas a condição não filtra nada. Sem cláusula `FOR`, vale também para escrita e exclusão.

Atenuante verificado: **o frontend não referencia nenhuma tabela** `wms_*` — busca por `wms_` em `src/` não retorna nada. São remanescentes de um módulo de almoxarifado que não está em uso.

IMPACTO

Leitura e escrita irrestritas em cinco tabelas por qualquer conta autenticada, incluindo `factory`. Hoje sem dado sensível em uso; o risco é a tabela voltar a ser usada com a política frouxa ainda no lugar.

CONDIÇÃO DE EXPLORABILIDADE

Qualquer conta autenticada. Sem impacto prático enquanto as tabelas seguirem vazias e sem uso.

CORREÇÃO

Se o módulo foi abandonado, remover as tabelas — código morto com porta aberta é pior que código morto. Se houver plano de retomá-lo, trocar a política pelo padrão do resto do sistema: `USING (public.is_admin_or_manager())`.

C2 — Permissão definida no navegador

ALTA **#2 — Papel de fábrica lê indicadores financeiros por RPC**

`supabase/fix_liquidez_dias_e_ciclo.sql:9-16, 88-89`

```
CREATE OR REPLACE FUNCTION public.get_liquidity_kpis(
  p_start date, p_end date
) RETURNS TABLE(pmr_days, pmp_days, cce_days, coverage_days, avg_daily_outflow)
LANGUAGE plpgsql
SECURITY DEFINER            -- ignora RLS
...
GRANT EXECUTE ON FUNCTION public.get_liquidity_kpis(date, date) TO authenticated;
```

POR QUE É EXPLORÁVEL

É a **única** função financeira das 55 auditadas que não chama `assert_finance_access()`. Sendo `SECURITY DEFINER`, ela roda com os privilégios do dono e **ignora a RLS** que protege `financial_obligations` e `obligation_payments`.

O frontend esconde o Financeiro do papel `factory` (`src/App.tsx:99, 126, 186`), mas isso é decisão de navegador. A RPC está concedida a `authenticated`, sem distinção de papel:
`await supabase.rpc('get_liquidity_kpis',{p_start:'2020-01-01',p_end:'2030-01-01'})`

IMPACTO

Prazo médio de recebimento e de pagamento, ciclo de conversão de caixa, saída média diária e dias de cobertura — o retrato financeiro da empresa para quem só deveria ver o chão de fábrica.

CONDIÇÃO DE EXPLORABILIDADE

Conta autenticada com papel `factory`. Expõe agregados, não linhas.

CORREÇÃO

Acrescentar `PERFORM public.assert_finance_access();` como primeira instrução do corpo, como já fazem `set_bank_balance`, `pay_obligation`, `reverse_payment` e `recalculate_all_bank_balances`.

ALTA **#3 — Papel de fábrica altera qualquer coluna de um pedido, inclusive o valor**

`supabase/security_hardening_single_tenant.sql:169-173`

```
CREATE POLICY single_tenant_factory_update_orders
  ON public.orders
  FOR UPDATE TO authenticated
  USING (public.is_factory())
  WITH CHECK (public.is_factory());
```

POR QUE É EXPLORÁVEL

A política autoriza UPDATE na tabela inteira, sem recortar colunas. A intenção legítima é permitir que a fábrica mova o pedido de setor no kanban — e é só isso que o frontend oferece. Mas `orders` guarda, nas mesmas linhas, `total_amount`, `amount_paid`, `discount`, `status` e `client_id` (`supabase/schema_botezini.sql:347-357`).

```
await supabase.from('orders').update({total_amount:0}).eq('id', qualquerPedido)
```

IMPACTO

Adulteração do valor de venda, do desconto e do valor já pago; troca do cliente do pedido; marcação de pedidos como entregues. Contamina faturamento, margem e as contas a receber geradas a partir do pedido.

CONDIÇÃO DE EXPLORABILIDADE

Conta autenticada com papel `factory`. Explorável direto pelo cliente Supabase.

CORREÇÃO

PostgreSQL não recorta colunas dentro de uma política. Dois caminhos:

(a) Um gatilho `BEFORE UPDATE` em `orders` que, quando `is_factory()`, rejeite qualquer alteração fora de `production_stage`, `items` e `updated_at`.

(b) Remover a política e mover o avanço de etapa para uma RPC `SECURITY DEFINER` que aceite só o pedido e o setor — no mesmo molde de `log_production`. É a opção mais limpa, porque a superfície passa a ser um argumento em vez de uma tabela.

C3 — IDOR

MÉDIA **#5 — Anexo sem validação de tipo, e URL assinada para caminho arbitrário**

`src/repositories/attachmentRepository.ts:9-14, 20-27`

```
const ext = file.name.split('.').pop();          // linha 9 — vem do usuário
const fileName = `${user.id}/${folder}/${Date.now()}_...${ext}`;
await supabase.storage.from('financial-attachments').upload(fileName, file);

async getAttachmentUrl(path: string) {           // linha 20 — path do chamador
  ...createSignedUrl(path, 3600);
}
```

POR QUE É EXPLORÁVEL

Sem validação de tipo: a extensão sai do nome do arquivo, sem lista de permitidos e sem conferir o MIME. Um `.svg` ou um HTML disfarçado é aceito, e o visualizador o entrega em `<iframe src={url}>` ou num link direto (`src/components/finance/AttachmentViewer.tsx:54, 61`). Script embutido executa na origem do Storage — outro domínio que não o do app, o que limita o dano, mas é XSS armazenado numa origem confiável.

Caminho arbitrário: `getAttachmentUrl` assina o que receber. Como o caminho começa com o id de quem enviou, informar o id de outra pessoa é IDOR direto — *a menos que* a política do bucket barre. **Não há política de Storage versionada em nenhuma das 46 migrações**, então essa proteção não pode ser confirmada a partir do código.

IMPACTO

Leitura de anexos financeiros de terceiros (notas, comprovantes) se a política do bucket for permissiva; execução de script numa origem confiável via arquivo enviado.

CONDIÇÃO DE EXPLORABILIDADE

O IDOR depende da política do bucket `financial-attachments`, que não está no repositório e precisa ser conferida no painel do Supabase. A falta de validação de tipo é incondicional.

CORREÇÃO

Restringir tipo e tamanho no envio (lista de permitidos por MIME **e** extensão) e passar `contentType` explícito no upload. Versionar a política do bucket numa migração, restringindo por prefixo de caminho — `(storage.foldername(name))[1] = auth.uid()::text` — ou por papel, conforme a regra do negócio. Servir anexo com `Content-Disposition: attachment`.

C4 — Chaves expostas

MÉDIA **#4 — Credenciais no histórico do Git**

`.env, .env.local, .env.vercel — commits ea41fd6, b1dd8b4, 51f0dfc`

```
VERCEL_OIDC_TOKEN="eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIs..."   # truncado
VITE_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  # 2 projetos
```

POR QUE É EXPLORÁVEL

Três arquivos de ambiente foram versionados e depois removidos do rastreio, mas **permanecem recuperáveis no histórico**: qualquer pessoa com acesso ao repositório (inclusive um clone antigo) lê os valores com `git log -p`.

O `VERCEL_OIDC_TOKEN` é credencial real de deploy — os dois exemplares encontrados já expiraram (`exp` de ~12 h), o que reduz o dano *destes*, mas confirma que o fluxo de trabalho comita segredo.

As `VITE_SUPABASE_ANON_KEY` (de dois projetos, `diajunbzicuruxqyfngm` e `ggrmdfagierzntrtzpey`) são **públicas por desenho** — vão no bundle de qualquer forma. Não são vazamento; o que elas revelam é a identidade dos projetos, e o quanto a segurança depende inteiramente da RLS estar correta.

IMPACTO

Tokens de deploy expirados, portanto sem impacto direto hoje. O risco é o padrão: o próximo segredo comitado pode ser de vida longa. Os refs de projeto expostos tornam o achado nº 1 diretamente endereçável.

CONDIÇÃO DE EXPLORABILIDADE

Acesso de leitura ao repositório ou a qualquer clone.

CORREÇÃO

Rotacionar o token da Vercel por precaução. Reescrever o histórico (`git filter-repo --path .env --path .env.local --path .env.vercel --invert-paths`) e forçar o envio, avisando quem tiver clone — ou, se reescrever não for viável, registrar a decisão de aceitar o risco por os tokens já terem expirado. O `.gitignore` já cobre `.env*` desde `51f0dfc`; acrescentar um hook de pré-commit que recuse padrões de segredo.

C5 — Inputs sem tratamento (XSS)

Nenhum achado nesta categoria. Ver a linha correspondente nos pontos fortes.

Tabela de achados

CRÍTICA	C1	`supabase/schema_live.sqlnpx:2103 (dump de produção)`	Qualquer usuário pode se promover a administrador
ALTA	C2	`supabase/fix_liquidez_dias_e_ciclo.sql: 9-16, 88-89`	Papel de fábrica lê indicadores financeiros por RPC
ALTA	C2	`supabase/security_hardening_single_tenant.sql:169-173`	Papel de fábrica altera qualquer coluna de um pedido, inclusive o valor
MÉDIA	C4	`.env, .env.local, .env.vercel — commits ea41fd6, b1dd8b4, 51f0dfc`	Credenciais no histórico do Git
MÉDIA	C3	`src/repositories/attachmentRepository.ts:9-14, 20-27`	Anexo sem validação de tipo, e URL assinada para caminho arbitrário

BAIXA	C1	`supabase/schema_live.sqlnpx (dump de produção) — 5 políticas`	Tabelas de almoxarifado abertas a qualquer usuário autenticado

Recomendações priorizadas

P1	**Derrubar a política `owner` de `profiles` e auditar políticas duplicadas em todas as tabelas**	É uma linha de SQL e fecha a escalada de privilégio. Na mesma sessão, listar `pg_policies` inteiro e derrubar toda política do modelo antigo por dono que tenha sobrevivido — o problema do achado nº 1 pode estar repetido em silêncio em outras tabelas.
P2	**Fechar as duas portas do papel de fábrica para o dinheiro**	Acrescentar `assert_finance_access()` em `get_liquidity_kpis` e restringir por gatilho as colunas que `factory` pode alterar em `orders`. Juntas, são as duas exceções à regra que o resto do sistema já segue.
P3	**Versionar a política do bucket e validar o tipo do anexo**	Trazer a autorização do Storage para dentro de `supabase/`, onde é revisável, e restringir o envio por MIME e extensão.
P4	**Rotacionar o token da Vercel e barrar segredo no pré-commit**	Os tokens encontrados já expiraram; o que precisa mudar é o hábito que os comitou.
P5	**Remover as tabelas `wms_*` ou alinhá-las ao padrão**	Cinco tabelas sem uso com política que não filtra nada. Remover é melhor que corrigir: não há código que dependa delas.

Issues para o GitHub

Uma issue por achado, em Markdown, pronta para copiar e colar. Os blocos são delimitados para que a cópia não arraste texto do relatório junto.

--- ISSUE 1 ---

```
## [Segurança] Qualquer usuário pode se promover a administrador

**Labels:** `security`, `severidade:crítica`, `c1`

### Problema
A política não tem cláusula `FOR`, então vale para SELECT, INSERT, UPDATE e DELETE. A única condição de
escrita é que a linha seja a sua — a coluna `role` fica sem restrição nenhuma.

A migração `perfil_nome_e_foto.sql:39-47` cria uma política correta, que prende o cargo no `WITH CHECK`.
 Mas **políticas de RLS são combinadas por OU**: basta uma permitir. A política `owner` nunca é removida
 — nenhuma das 46 migrações a derruba, e `security_hardening_single_tenant.sql` só ACRESCENTA políticas.

Exploração, do console do navegador, com uma conta `factory`:
`await supabase.from('profiles').update({role:'admin'}).eq('id', meuId)`

A partir daí `is_admin_or_manager()` passa a devolver verdadeiro, e todo o Financeiro, todos os dados e
todas as RPCs se abrem.

### Evidência
`supabase/schema_live.sqlnpx:2103 (dump de produção)`

```sql
CREATE POLICY "owner" ON "public"."profiles"
  USING (("auth"."uid"() = "id"))
  WITH CHECK (("auth"."uid"() = "id"))
```

Impacto
Escalada completa de privilégio, do papel mais baixo ao de administrador. Dá acesso a saldos bancários,
contas a pagar e receber, cadastro de clientes e à capacidade de apagar dados.

Condição de explorabilidade
Requer uma conta autenticada de qualquer papel. Nenhuma configuração especial. A chave anônima necessári
a está publicada no bundle, por desenho.

Sugestão de correção
Derrubar a política antiga e verificar que nenhuma outra permissiva sobrou:

`DROP POLICY IF EXISTS "owner" ON public.profiles;`

Depois conferir com `SELECT policyname, cmd, qual, with_check FROM pg_policies WHERE tablename='profiles
';` — devem sobrar apenas as duas de `perfil_nome_e_foto.sql` e a de admin/manager.

Como defesa em profundidade, um gatilho `BEFORE UPDATE` em `profiles` que rejeite mudança de `role` quan
do `auth.uid() = NEW.id`: a promoção passa a exigir outra pessoa, independentemente de qual política ven
ha a existir no futuro.

Critérios de aceite
- [ ] A política `owner` não aparece mais em `pg_policies` para `profiles`
- [ ] Uma conta `factory` recebe erro ao tentar gravar `role='admin'` em si mesma
- [ ] A mesma conta continua conseguindo alterar o próprio nome e a própria foto
- [ ] Um administrador continua conseguindo alterar o papel de outra pessoa
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
## [Segurança] Papel de fábrica lê indicadores financeiros por RPC

**Labels:** `security`, `severidade:alta`, `c2`

### Problema
É a **única** função financeira das 55 auditadas que não chama `assert_finance_access()`. Sendo `SECURIT
Y DEFINER`, ela roda com os privilégios do dono e **ignora a RLS** que protege `financial_obligations` e
 `obligation_payments`.

O frontend esconde o Financeiro do papel `factory` (`src/App.tsx:99, 126, 186`), mas isso é decisão de n
avegador. A RPC está concedida a `authenticated`, sem distinção de papel:
`await supabase.rpc('get_liquidity_kpis',{p_start:'2020-01-01',p_end:'2030-01-01'})`

### Evidência
`supabase/fix_liquidez_dias_e_ciclo.sql:9-16, 88-89`

```sql
CREATE OR REPLACE FUNCTION public.get_liquidity_kpis(
  p_start date, p_end date
) RETURNS TABLE(pmr_days, pmp_days, cce_days, coverage_days, avg_daily_outflow)
LANGUAGE plpgsql
SECURITY DEFINER            -- ignora RLS
...
GRANT EXECUTE ON FUNCTION public.get_liquidity_kpis(date, date) TO authenticated;
```

Impacto
Prazo médio de recebimento e de pagamento, ciclo de conversão de caixa, saída média diária e dias de cob
ertura — o retrato financeiro da empresa para quem só deveria ver o chão de fábrica.

Condição de explorabilidade
Conta autenticada com papel `factory`. Expõe agregados, não linhas.

Sugestão de correção
Acrescentar `PERFORM public.assert_finance_access();` como primeira instrução do corpo, como já fazem `s
et_bank_balance`, `pay_obligation`, `reverse_payment` e `recalculate_all_bank_balances`.

Critérios de aceite
- [ ] `get_liquidity_kpis` chama `assert_finance_access()` antes de qualquer leitura
- [ ] Conta `factory` recebe 'Acesso financeiro negado' ao chamar a RPC
- [ ] O painel financeiro de admin/manager continua carregando os indicadores
- [ ] Uma consulta a `pg_proc` confirma que nenhuma outra função SECURITY DEFINER financeira ficou sem a
 checagem
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
## [Segurança] Papel de fábrica altera qualquer coluna de um pedido, inclusive o valor

**Labels:** `security`, `severidade:alta`, `c2`

### Problema
A política autoriza UPDATE na tabela inteira, sem recortar colunas. A intenção legítima é permitir que a
 fábrica mova o pedido de setor no kanban — e é só isso que o frontend oferece. Mas `orders` guarda, nas
 mesmas linhas, `total_amount`, `amount_paid`, `discount`, `status` e `client_id` (`supabase/schema_bote
zini.sql:347-357`).

`await supabase.from('orders').update({total_amount:0}).eq('id', qualquerPedido)`

### Evidência
`supabase/security_hardening_single_tenant.sql:169-173`

```sql
CREATE POLICY single_tenant_factory_update_orders
  ON public.orders
  FOR UPDATE TO authenticated
  USING (public.is_factory())
  WITH CHECK (public.is_factory());
```

Impacto
Adulteração do valor de venda, do desconto e do valor já pago; troca do cliente do pedido; marcação de p
edidos como entregues. Contamina faturamento, margem e as contas a receber geradas a partir do pedido.

Condição de explorabilidade
Conta autenticada com papel `factory`. Explorável direto pelo cliente Supabase.

Sugestão de correção
PostgreSQL não recorta colunas dentro de uma política. Dois caminhos:

(a) Um gatilho `BEFORE UPDATE` em `orders` que, quando `is_factory()`, rejeite qualquer alteração fo
ra de `production_stage`, `items` e `updated_at`.

(b) Remover a política e mover o avanço de etapa para uma RPC `SECURITY DEFINER` que aceite só o ped
ido e o setor — no mesmo molde de `log_production`. É a opção mais limpa, porque a superfície passa a se
r um argumento em vez de uma tabela.

Critérios de aceite
- [ ] Conta `factory` não consegue alterar `total_amount`, `discount`, `amount_paid` nem `client_id`
- [ ] A mesma conta continua movendo cartões entre setores no kanban
- [ ] Existe teste ou consulta que comprove a recusa
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

```
## [Segurança] Credenciais no histórico do Git

**Labels:** `security`, `severidade:média`, `c4`

### Problema
Três arquivos de ambiente foram versionados e depois removidos do rastreio, mas **permanecem recuperáveis no histórico**: qualquer pessoa com acesso ao repositório (inclusive um clone antigo) lê os valores com `git log -p`.

O `VERCEL_OIDC_TOKEN` é credencial real de deploy — os dois exemplares encontrados já expiraram (`exp` de ~12 h), o que reduz o dano _destes_, mas confirma que o fluxo de trabalho comita segredo.

As `VITE_SUPABASE_ANON_KEY` (de dois projetos, `diajunbzicuruxqyfngm` e `ggrmdfagierzntrtzpey`) são **públicas por desenho** — vão no bundle de qualquer forma. Não são vazamento; o que elas revelam é a identidade dos projetos, e o quanto a segurança depende inteiramente da RLS estar correta.

### Evidência
`.env, .env.local, .env.vercel — commits ea41fd6, b1dd8b4, 51f0dfc`

```sql
VERCEL_OIDC_TOKEN="eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCIs..."   # truncado
VITE_SUPABASE_ANON_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  # 2 projetos
```

Impacto
Tokens de deploy expirados, portanto sem impacto direto hoje. O risco é o padrão: o próximo segredo comitado pode ser de vida longa. Os refs de projeto expostos tornam o achado nº 1 diretamente endereçável.

Condição de explorabilidade
Acesso de leitura ao repositório ou a qualquer clone.

Sugestão de correção
Rotacionar o token da Vercel por precaução. Reescrever o histórico (`git filter-repo --path .env --path .env.local --path .env.vercel --invert-paths`) e forçar o envio, avisando quem tiver clone — ou, se reescrever não for viável, registrar a decisão de aceitar o risco por os tokens já terem expirado. O `.gitignore` já cobre `.env*` desde `51f0dfc`; acrescentar um hook de pré-commit que recuse padrões de segredo.

Critérios de aceite
- [ ] Token da Vercel rotacionado
- [ ] `git log --all -S 'VERCEL_OIDC_TOKEN'` não retorna nada, OU a exceção está registrada por escrito
- [ ] Hook de pré-commit recusa arquivos `.env` e cadeias no formato JWT
```

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

```
## [Segurança] Anexo sem validação de tipo, e URL assinada para caminho arbitrário

**Labels:** `security`, `severidade:média`, `c3`

### Problema
**Sem validação de tipo:** a extensão sai do nome do arquivo, sem lista de permitidos e sem conferir o MIME. Um `.svg` ou um HTML disfarçado é aceito, e o visualizador o entrega em `<iframe src={url}>` ou num link direto (`src/components/finance/AttachmentViewer.tsx:54, 61`). Script embutido executa na origem do Storage — outro domínio que não o do app, o que limita o dano, mas é XSS armazenado numa origem confiável.

**Caminho arbitrário:** `getAttachmentUrl` assina o que receber. Como o caminho começa com o id de quem enviou, informar o id de outra pessoa é IDOR direto — _a menos que_ a política do bucket barre. **Não há política de Storage versionada em nenhuma das 46 migrações**, então essa proteção não pode ser confirmada a partir do código.

### Evidência
`src/repositories/attachmentRepository.ts:9-14, 20-27`

```sql
const ext = file.name.split('.').pop();          // linha 9 — vem do usuário
const fileName = `${user.id}/${folder}/${Date.now()}_...${ext}`;
await supabase.storage.from('financial-attachments').upload(fileName, file);

async getAttachmentUrl(path: string) {           // linha 20 — path do chamador
  ...createSignedUrl(path, 3600);
}
```

Impacto
Leitura de anexos financeiros de terceiros (notas, comprovantes) se a política do bucket for permissiva; execução de script numa origem confiável via arquivo enviado.

Condição de explorabilidade
O IDOR depende da política do bucket `financial-attachments`, que não está no repositório e precisa ser conferida no painel do Supabase. A falta de validação de tipo é incondicional.

Sugestão de correção
Restringir tipo e tamanho no envio (lista de permitidos por MIME **e** extensão) e passar `contentType` explícito no upload. Versionar a política do bucket numa migração, restringindo por prefixo de caminho — `(storage.foldername(name))[1] = auth.uid()::text` — ou por papel, conforme a regra do negócio. Servir anexo com `Content-Disposition: attachment`.

Critérios de aceite
- [ ] Envio recusa extensão fora da lista permitida (teste com `.svg` e `.html`)
- [ ] A política do bucket está numa migração dentro de `supabase/`
- [ ] Usuário A recebe erro ao pedir URL assinada de um caminho do usuário B
- [ ] Anexo abre como download, não renderizado inline
```

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

```
## [Segurança] Tabelas de almoxarifado abertas a qualquer usuário autenticado

**Labels:** `security`, `severidade:baixa`, `c1`

### Problema
`auth.uid() IS NOT NULL` é verdadeiro para **todo** usuário logado. A RLS está habilitada — o que faz a
tabela parecer protegida numa auditoria superficial — mas a condição não filtra nada. Sem cláusula `FOR`
, vale também para escrita e exclusão.

Atenuante verificado: **o frontend não referencia nenhuma tabela `wms_*`** — busca por `wms_` em `src/`
não retorna nada. São remanescentes de um módulo de almoxarifado que não está em uso.

### Evidência
`supabase/schema_live.sqlnpx (dump de produção) — 5 políticas`

```sql
CREATE POLICY "authenticated" ON "public"."wms_items"          USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_categories"     USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_item_variations" USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_stock_items"    USING ((auth.uid() IS NOT NULL))
CREATE POLICY "authenticated" ON "public"."wms_stock_movements" USING ((auth.uid() IS NOT NULL))
```

Impacto
Leitura e escrita irrestritas em cinco tabelas por qualquer conta autenticada, incluindo `factory`. Hoje
 sem dado sensível em uso; o risco é a tabela voltar a ser usada com a política frouxa ainda no lugar.

Condição de explorabilidade
Qualquer conta autenticada. Sem impacto prático enquanto as tabelas seguirem vazias e sem uso.

Sugestão de correção
Se o módulo foi abandonado, remover as tabelas — código morto com porta aberta é pior que código morto.
Se houver plano de retomá-lo, trocar a política pelo padrão do resto do sistema: `USING (public.is_admin
_or_manager())`.

Critérios de aceite
- [ ] As tabelas `wms_*` foram removidas OU passaram a usar `is_admin_or_manager()`
- [ ] Nenhuma política do banco usa `auth.uid() IS NOT NULL` como única condição
```

--- FIM ISSUE 6 ---